DOCKER FUNDAMENTALS

Understanding Docker Concepts

Containers, Images, Volumes, Networking & Docker Compose — explained with real-world analogies

A Beginner's Guide

THE PROBLEM

Why Do We Need Docker?

"It works on my machine..."

! **Different environments**

Your laptop, a teammate's PC, and the server all have different OS versions, libraries, and configs.

! **Manual setup pain**

Installing the right Python version, dependencies, and databases by hand is slow and error-prone.

! **Inconsistent results**

Code that runs perfectly for you crashes elsewhere — wasting hours of debugging.

The Shipping Container Analogy

Before standard shipping containers, cargo was packed by hand — every ship loaded differently. The standard steel container changed everything: any container fits any ship, truck, or crane, anywhere in the world.

Docker does this for software: your app + everything it needs, packaged once, running identically everywhere.

CORE CONCEPT

Containers vs. Virtual Machines

Virtual Machine

Like a detached house — its own plumbing, wiring, and foundation (a full guest OS).

- Boots in minutes
- Gigabytes in size
- Full OS-level isolation
- Runs via a hypervisor

Container

Like an apartment in a shared building — its own rooms, but shares the building's foundation (host OS kernel).

- Starts in seconds
- Megabytes in size
- Process-level isolation
- Runs via Docker Engine

Core Docker Commands

`docker pull <image>`	Download an image from Docker Hub
`docker run <image>`	Create and start a container from an image
`docker ps / docker ps -a`	List running / all containers
`docker exec -it <name> bash`	Open a shell inside a running container
`docker logs <name>`	View a container's output
`docker stop / rm <name>`	Stop and remove a container

Volumes: Making Data Survive

Storage Locker Analogy

Containers are disposable, like a hotel room — when you check out, everything inside resets. A volume is a storage locker outside the room: it survives even if the container is deleted and rebuilt.

```
docker volume create my_data

docker run -it \
  -v my_data:/app/data \
  ubuntu bash

# data in /app/data persists
# even after the container
# is removed
```

Volumes = data that outlives the container.

Networking: Containers Talking to Each Other

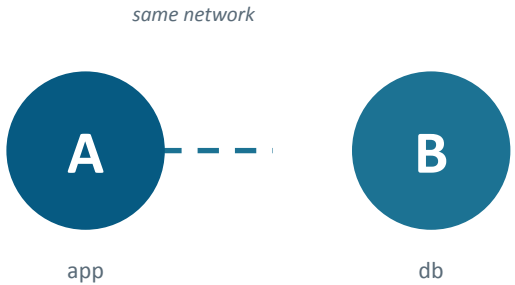
Same-Floor Apartments

By default, containers are isolated strangers in different buildings. A Docker network puts them on the same floor, with each apartment door labeled by name — so they can call each other by name instead of an IP address.

```
docker network create my_net

docker run --network my_net ...
```

→ containers now reach each other by container name (e.g. ping db)



The Dockerfile: A Recipe for Your App

A Dockerfile is a recipe card. Just like a recipe lists ingredients and steps in order, a Dockerfile lists a base image and the steps to build your app's image. The result — the image — always turns out the same, no matter who builds it.

```
docker build -t my-app .

docker run --rm my-app
```

build reads the recipe and bakes the image → run serves the finished dish

```
FROM python:3.11-slim                    // base ingredient

WORKDIR /app                             // the kitchen counter

COPY requirements.txt .                  // ingredient list first

RUN pip install -r requirements.txt      // install deps (cached!)

COPY . .                                 // rest of the app

CMD ["python", "pipeline.py"]            // final step to run
```

Why COPY requirements first? Docker caches each step — if code changes but dependencies don't, the install step is reused instead of repeated.

Multi-Stage Builds with uv

Prep Kitchen vs. Serving Kitchen

A prep kitchen is messy — sacks of flour, extra tools everywhere. But the dish served has none of that mess. A multi-stage build uses one throwaway stage to install dependencies with uv (a very fast installer), then copies only the finished result into a clean, lightweight final image.

Result: smaller, faster, more secure images — no leftover build tools or caches in the final image.

```
# --- Stage 1: prep kitchen ---
FROM python:3.11-slim AS builder
RUN pip install uv
WORKDIR /app
COPY requirements.txt .
RUN uv pip install --system \
    --target=/app/deps \
    -r requirements.txt

# --- Stage 2: serving kitchen ---
FROM python:3.11-slim
COPY --from=builder /app/deps ...
COPY . .
CMD ["python", "pipeline.py"]
```

PostgreSQL in Docker + pgAdmin

PostgreSQL Container

```
docker run -d \
  -e POSTGRES_USER=root \
  -e POSTGRES_PASSWORD=root \
  -e POSTGRES_DB=ny_taxi \
  -v pg_data:/var/lib/postgresql/data \
  -p 5432:5432 postgres:16
```

The -e flags set the database's "employee ID card": username, password, and which database to create.

pgAdmin = the Dashboard

Postgres is the engine — powerful, but you don't want to touch pistons directly. pgAdmin is the dashboard: gauges and a steering wheel for a friendly view of what's happening under the hood.

⚠️ *Two separate docker run containers can't reach each other by localhost — they need to be on the same Docker network.*

Docker Compose: One Blueprint

Instead of manually building each apartment (container) and wiring them together, docker-compose.yml is one blueprint describing the whole complex — every service, the shared hallways (network), and storage lockers (volumes) — all at once.

```
docker compose up -d

docker compose down
```

Compose auto-creates a shared network — services reach each other by their service name (e.g. pgdatabase), not localhost.

```
services:
  pgdatabase:
    image: postgres:16
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: root
      POSTGRES_DB: ny_taxi
    volumes:
      - pg_data:/var/lib/...
    ports:
      - "5432:5432"

  pgadmin:
    image: dpage/pgadmin4
    ports:
      - "8081:80"
```

PUTTING IT TOGETHER

Project: NY Taxi Data Ingestion Pipeline

1

Read the CSV in chunks

pandas reads 100k rows at a time — like carrying groceries in bags, not all at once.

2

Load into Postgres

SQLAlchemy writes each chunk to a table running inside a Dockerized Postgres.

3

Run it all with Compose

Postgres + pgAdmin + the ingestion script all defined in one docker-compose.yml.

```
Verify: SELECT COUNT(*) FROM yellow_taxi_trips;  →  millions of rows, loaded reliably.
```

RECAP

Key Takeaways

- **Containers:** Lightweight, portable packages of your app + everything it needs to run.
- **Images vs Containers:** An image is the recipe/blueprint; a container is a running instance of it.
- **Volumes:** Keep data alive even after a container is deleted.
- **Networks:** Let containers discover and talk to each other by name.
- **Docker Compose:** Define and run multi-container apps with a single YAML file.

"No more 'it works on my machine.'"